Part 2 Report:

**1. Which insertion strategy produces the tallest BST? Explain why.**

- From what I am looking at in the results, It seems like the ascending order is producing the tallest BST. Being in a straightforward order, it would be expected that the tree would increase in size.

**2. How does the height of the BST change as N increases for each strategy? Relate your observations to the theoretical time complexity of BST operations.**

- I believe that the way that the height changes is based on the order provided by the strategy that you used. For a balanced strategy, the height might stay at a consistent height due to the order that you put on it.

**3. Compare the average successful search comparisons across the three strategies. How closely do these results correspond to the tree heights?**

- I unfortunately was not able to produce this part of the experiment in this paper due to time constraints. I might still experiment on the source code even if it is late.

**4. Is a BST constructed using random insertion necessarily balanced? Explain based on your experiments.**

- Random insertions can be balanced depending on what order of numbers. Either it could be at the worst case scenario possible, or it will be so unnecessarily balanced that the tree might look confusing.

Part 3 Report:

I was not able to move on to part 3 due to time constraints. However, I might still go through with the experiments anyways even if it's too late. So here's my AI report:

AI Report:

I've used AI a few times during this process.

1. The first time I used AI was when I was getting errors that said 'NoneType object is not callable'. So I prompted a screenshot and the AI told me that I had some small variable logic errors which just resulted in changing a single word in the code. It was an insert code that was in a separate BST code for 2 and 3.

2. The second time was the same case but with the depth definition. I ended up changing that value as well.